

Practical 12

Task 1

Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>
#define ITEMS 100000
int *buffer;
int buffer_size;
int in = 0;
int out = 0;
sem_t empty;
sem_t full;
pthread_mutex_t mutex;
void *producer(void *arg)
{
    for (int i = 0; i < ITEMS; i++)
    {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = i;
        in = (in + 1) % buffer_size;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
    return NULL;
}
void *consumer(void *arg)
{
    long long sum = 0;
```

```

{
    long long sum = 0;
    for (int i = 0; i < ITEMS; i++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        out = (out + 1) % buffer_size;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sum += item;
    }
    printf("Consumed sum = %lld\n", sum);
    return NULL;
}

void test_buffer_size(int size)
{
    pthread_t producer_thread;
    pthread_t consumer_thread;
    buffer_size = size;
    buffer = malloc(buffer_size * sizeof(int));
    if (buffer == NULL)
    {
        perror("Memory allocation failed");
        exit(1);
    }
    in = 0;
    out = 0;
    sem_init(&empty, 0, buffer_size);

```

```

{
    long long sum = 0;
    for (int i = 0; i < ITEMS; i++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        out = (out + 1) % buffer_size;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sum += item;
    }
    printf("Consumed sum = %lld\n", sum);
    return NULL;
}

void test_buffer_size(int size)
{
    pthread_t producer_thread;
    pthread_t consumer_thread;
    buffer_size = size;
    buffer = malloc(buffer_size * sizeof(int));
    if (buffer == NULL)
    {
        perror("Memory allocation failed");
        exit(1);
    }
    in = 0;
    out = 0;
    sem_init(&empty, 0, buffer_size);

```

```

sem_init(&empty, 0, buffer_size);
sem_init(&full, 0, 0);
pthread_mutex_init(&mutex, NULL);
clock_t start = clock();
pthread_create(
    &producer_thread,
    NULL,
    producer,
    NULL
);
pthread_create(
    &consumer_thread,
    NULL,
    consumer,
    NULL
);
pthread_join(producer_thread, NULL);
pthread_join(consumer_thread, NULL);
clock_t end = clock();
double time_taken =
    (double)(end - start) / CLOCKS_PER_SEC;
double throughput =
    ITEMS / time_taken;
printf("\nBuffer Size = %d\n", buffer_size);
printf("Execution Time = %.6f seconds\n",
    time_taken);
printf("Throughput = %.2f items/second\n",
    throughput);
sem_destroy(&empty);

```

```

    time_taken);
printf("Throughput = %.2f items/second\n",
    throughput);
sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);
free(buffer);
}
int main()
{
    printf("Producer-Consumer Problem\n");
    printf("Total Items = %d\n\n", ITEMS);
    test_buffer_size(5);
    test_buffer_size(10);
    test_buffer_size(50);
    test_buffer_size(100);
    return 0;
}

```

```
root@Tejasri:~# nano posix.c
root@Tejasri:~# gcc posix.c -o posix
root@Tejasri:~# ./posix
Producer-Consumer Problem
Total Items = 100000

Consumed sum = 4999950000

Buffer Size = 5
Execution Time = 0.505367 seconds
Throughput = 197876.00 items/second
Consumed sum = 4999950000

Buffer Size = 10
Execution Time = 0.262061 seconds
Throughput = 381590.55 items/second
Consumed sum = 4999950000

Buffer Size = 50
Execution Time = 0.059483 seconds
Throughput = 1681152.60 items/second
Consumed sum = 4999950000

Buffer Size = 100
Execution Time = 0.055294 seconds
Throughput = 1808514.49 items/second
root@Tejasri:~# |
```

Task 2

Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
pthread_mutex_t resource1;
pthread_mutex_t resource2;
void *thread1_function(void *arg)
{
    printf("Thread 1: Trying to lock Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 1: Locked Resource 1\n");
    sleep(1);
    printf("Thread 1: Trying to lock Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 1: Locked Resource 2\n");
    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);
    return NULL;
}
void *thread2_function(void *arg)
{
    printf("Thread 2: Trying to lock Resource 2\n");
    pthread_mutex_lock(&resource2);
    printf("Thread 2: Locked Resource 2\n");
    sleep(1);
    printf("Thread 2: Trying to lock Resource 1\n");
    pthread_mutex_lock(&resource1);
    printf("Thread 2: Locked Resource 1\n");
    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);
}

```

```

    pthread_mutex_unlock(&resource2);
    return NULL;
}
int main()
{
    pthread_t thread1, thread2;
    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);
    printf("Starting deadlock demonstration...\n\n");
    pthread_create(
        &thread1,
        NULL,
        thread1_function,
        NULL
    );
    pthread_create(
        &thread2,
        NULL,
        thread2_function,
        NULL
    );
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);
    printf("\nProgram completed.\n");
    return 0;
}

```

```
root@Tejasri:~# nano deadlock.c
root@Tejasri:~# gcc deadlock.c -o deadlock
root@Tejasri:~# ./deadlock
Starting deadlock demonstration...

Thread 1: Trying to lock Resource 1
Thread 1: Locked Resource 1
Thread 2: Trying to lock Resource 2
Thread 2: Locked Resource 2
Thread 1: Trying to lock Resource 2
Thread 2: Trying to lock Resource 1
^C
root@Tejasri:~# |
```